ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we would like to express our sincere gratitude to our guide and major project coordinator, **Prof. Ajay Shastry C.G.** for providing excellent guidance, encouragement and inspiration throughout the project work. Without his invaluable guidance, this work would never have been a successful one.

We would like to express our sincere gratitude to the Head of the Department of Artificial Intelligence & Machine Learning, **Dr. Radhika Shetty D S**, for her guidance and inspiration. We would like to thank our Principal, **Dr. Mahesh Prasanna K**, for providing all the facilities and a proper environment to work in the college campus.

We would like to thank our management for providing the necessary infrastructure to carry out the project work.

We are thankful to all the teaching and non-teaching staff members of the Artificial Intelligence & Machine Learning Department for their help and needed support rendered throughout the project.

ABSTRACT

Stuttering is a speech disorder that disrupts the natural flow of communication through involuntary repetitions, prolongations, blocks, and pauses in speech. Nearly 1% of people worldwide are impacted, and it can significantly affect social, professional, and personal life. Traditional diagnosis methods rely on manual assessment by speech-language pathologists, which are subjective, time-consuming, and not easily accessible to all individuals. To overcome these limitations, this project presents an intelligent and automated system that leverages deep learning to detect and classify various types of stuttering events in speech recordings.

The core of the system is a multi-model deep learning-based architecture, where five parallel binary classifiers are individually trained to detect distinct stutter types including sound repetitions, word repetitions, prolongations, blocks, and interjections. The input audio is preprocessed and converted into embeddings, which are then analyzed using a CNN and Transformer-based architecture to capture both local acoustic patterns and long-range temporal dependencies in speech. Each model processes these features independently and outputs probability-based predictions for specific dysfluencies. The system also includes a transcription module that generates timestamped output from the same audio input. The detected stutter timestamps are then hard-aligned with the transcription timestamps to identify the exact word, syllable, or speech segment associated with the stutter event. The system is integrated with a user-friendly PySide6 graphical interface that allows users to record or upload audio samples, visualize waveform and receive detailed analysis and classification reports.

By combining deep learning-based speech analysis with transcript-based timestamp alignment and an interactive application interface, this system provides a reliable and accessible tool for early stutter detection. It reduces reliance on manual evaluation, offers objective insights into speech fluency, and can serve as an assistive aid for both speech pathologists and individuals undergoing therapy. This project demonstrates how artificial intelligence and audio signal processing can be effectively utilized in the healthcare domain to enhance diagnostic accuracy, improve therapy outcomes, and promote speech fluency assessment at scale.

TABLE OF CONTENT

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
ALE	Activattion Likelihood Estimation
ASR	Automatic Speech Recognition
BERT	Bidirectional Encoder Representations from Transformers
Bi-LSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
EEG	Electroencephalography
fMRI	Functional Magnetic Resonance Imaging
GUI	Graphical User Interface
HMM	Hidden Markov Model
isWER	Intended Speech Word Error Rate
LLM	Large Langauge Model
LSTM	Long Short-Term Memory
MB	Multi-Branching
MC	Multi-Contextual
MFCC	Mel-Frequency Cepstral Coefficients
ML-CNN	Multi-Label Convolutional Neural Network
PCA	Principal Component Analysis
PyQT5	Python QT Framework
ReLU	Rectified Linear Unit
RNN	REcurrent Neural Network
Sep-28k	Stuttering Event Prediction Dataset (28,000 samples)
SLP	Speech-Language Pathologist
SOTA	State Of The Art
STFT	Short-Time Fourier Transform
STT	Speech-to-Text
SVM	Support Vector Machine
TTS	Text-to-Speech
UI	User Interface
VITS	Variational Inference With Adversarial Learning for end-to-end Text-to-Speech
WavLM	Wave Langauge Model
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

Speech fluency is an essential aspect of human communication, allowing individuals to express thoughts and emotions clearly. However, many people face interruptions in their speech flow due to a disorder known as stuttering. It is a common speech disorder characterized by involuntary repetitions, prolongations, or blocks during speech production. This condition affects individuals across all age groups and can significantly impact their confidence, communication ability, and social interactions.

Early identification of stuttering patterns is crucial for effective therapy and management. Traditionally, detection and assessment of stuttering rely on manual observation by speech-language pathologists, which can be subjective, time- consuming, and often inconsistent across experts. Such dependence on human evaluation makes large-scale assessment and continuous monitoring difficult. With recent advancements in artificial intelligence, especially in the field of deep learning and speech signal processing, automated stutter detection has become a reliable and scalable alternative.

The project aims to develop an intelligent system that can automatically detect and classify various types of stuttering in speech, including repetitions, prolongations, blocks, and fillers, using deep learning techniques. The input audio will be first preprocessed and converted into embeddings, which will be then analyzed using a CNN and Transformer-based architecture to learn both local acoustic patterns and long-range temporal dependencies that differentiate normal and dysfluent speech. Along with stutter classification, the same audio will be also passed through a transcription module, and the detected stutter timestamps will be hard-aligned with the transcript timestamps to identify the exact word, syllable, or speech segment where the dysfluency occurs. The final model will be integrated into a user-friendly interface that allows users to record or upload speech samples and receive real-time analysis, timestamped detection results, and classification output.

Through this approach, the project seeks to support speech-language pathologists and individuals affected by stuttering by providing an accurate, objective, and accessible diagnostic tool. It also contributes to the broader vision of integrating artificial intelligence into healthcare to enhance diagnosis, therapy, and quality of life.

1.2 Existing System

Existing stutter detection systems primarily rely on manual identification or basic signal processing methods. Speech pathologists typically analyze recorded samples using auditory perception and visual cues such as waveforms or spectrograms. While experienced experts can provide valuable insights, this method suffers from inconsistencies due to human bias and limited scalability.

Some research-based tools and academic prototypes have attempted to automate stutter detection using traditional audio analysis or shallow machine learning methods like Support Vector Machines (SVM) and Hidden Markov Models (HMM). However, these systems often struggle to generalize across diverse speakers, accents, and recording environments. Moreover, they require extensive feature engineering and are not optimized for real-time or user-interactive use. Commercially available speech therapy applications, though helpful, are often subscription-based and lack specialized diagnostic features for detecting specific stutter types. Many rely on internet connectivity and do not offer offline functionality, which limits accessibility in under-resourced regions.

These limitations highlight the need for an advanced, reliable, and accessible stutter detection system. A deep learning–based approach using Convolutional Neural Networks (CNNs) and audio spectrogram analysis can automatically learn

1.3 Proposed System

The proposed system introduces an intelligent and automated framework for detecting and classifying stuttering patterns from speech recordings. It will leverages deep learning, specifically a multi-model binary classification architecture, to identify various dysfluency types effectively. The workflow will begin with capturing or uploading speech recordings. The audio will be then pre-processed and converted into embeddings, which will be analyzed using a CNN and Transformer-based model to capture both local acoustic patterns and long-range temporal dependencies. These learned representations will be used to detect five independent stuttering behaviors: repetitions, prolongations, blocks, interjections, and word-level dysfluencies.

Along with stutter detection, the same audio will also passed through a transcription model to generate timestamped speech output. The detected stutter timestamps will be then hard-aligned with the transcript timestamps to identify the exact word, syllable, or speech segment associated with the dysfluency. Each model will output a probability score indicating the presence of a specific stutter class, and the results are compiled into a final classification report. The trained model will be integrated into a desktop-based PyQt5 interface, allowing users to record speech, upload audio files, and visualize results in real time. The interface will also provides playback controls and waveform displays for better interpretability. By combining advanced speech signal processing, deep learning-based classification, and timestamp alignment with transcription, this system aims to deliver a complete end-to-end solution that will not only detect stuttering automatically but also support clinicians and individuals in monitoring progress and planning effective therapy interventions.

1.4 Scope of the Project

This project focuses on developing a deep learning-based speech analysis system that etects and classifies stuttering behaviors from recorded or live speech input. The system caters to both research and clinical applications by providing accurate, objective, and real-time dysfluency detection.

The project scope includes:

- Designing and training a hybrid CNN + Transformer model to classify speech dysfluencies into five categories: prolongation, block, sound repetition, word repetition, and interjection.
- Implementing a speech representation pipeline using Wav2Vec embeddings to extract deep acoustic features from raw audio without manual feature engineering.
- Developing a temporal localization and hard-alignment pipeline to map detected stutter timestamps with transcription timestamps and identify the exact word, syllable, or speech segment affected.
- Creating a cross-platform, user-friendly GUI using PyQt5 that enables users to record, upload, analyze, and visualize speech data interactively.
- Generating analytical outputs including probability scores, timestamped detections, waveform/spectrogram annotations, and diagnostic summaries.
- Supporting both real-time audio capture and offline file-based analysis to accommodate users with varying technical resources.
- Providing an intelligent and accessible tool that supports automation of speech pathology assessments and assists in early identification, monitoring, and therapy tracking for individuals with stuttering.

1.5 Objectives of the Project

The primary goal of this project is to create an automated and accurate system for detecting and classifying stuttering in speech recordings using deep learning techniques. The key objectives are:

- **Speech Representation Pipeline**: To develop a robust pipeline using the Wave2Vec framework to extract deep, context-aware embeddings from raw audio without manual feature engineering.
- **Hybrid CNN + Transformer Architecture**: To design and train a model that leverages both local acoustic patterns and global temporal context for accurate stutter detection and classification.
- **Specialized Classifiers For Each Stutter Class**: To build individual models for Prolongation, Block, Sound Repetition, Word Repetition, and Interjection to maximize class-wise accuracy.
- **Temporal Localization Module**: To identify the exact timestamps of stuttering events, the affected region, and the duration and frequency of occurrences within a speech sample.

- **Visual Annotation System**: To mark stutter-indicative regions on the waveform/spectrogram for an interpretable and clinician-friendly analysis interface.

1.6 Organization of the Report

This report is organized into two chapters, each presenting a structured explanation of the project's goal and the research carried out to support the future implementation of the project.

- **Chapter 1** introduces the project, its motivation, objectives, and overall structure. It outlines the problem of stuttering, the need for automation, and the proposed system's benefits.
- **Chapter 2** presents the Literature Survey, reviewing previous work, existing speech analysis tools, and limitations in current methods.
- **Chapter 3** explains the System Requirements, including both hardware and software specifications, as well as functional and non-functional requirements.
- **Chapter 4** focuses on System Design, providing architectural diagrams, data flow representations, and a detailed explanation of the CNN-based classification process.

CHAPTER 2

LITERATURE SURVEY

P. Arbajian et al. [1] have proposed *"Effect of Speech Segment Samples Selection in Stutter Block Detection and Remediation."*

In this study the impact of different speech segment selection strategies on the accuracy of stutter block detection. The authors study the impact of analyzing the speech with different window sizes and positions, instead of analyzing the whole signal in the same way. This approach consists in extracting acoustic features from annotated speech samples, and employing machine learning classifiers to evaluate performance under different segmentation setups. In this study, the authors investigate the effect of temporal segmentation on the detection of stuttering events by comparing different windowing approaches. The results indicate that segment choice has a strong impact on accuracy. Some segment lengths are better to model dysfluency patterns, but bad segmentation can lose important temporal information and lead to less performance. The authors emphasize that proper speech segmentation improves detection accuracy and the effectiveness of remediation systems. This study is relevant to the proposed system as it emphasizes the importance of preprocessing, in particular optimized speech segmentation. It facilitates the employment of well-structured input representations, e.g., segmented spectrograms, to enhance the performance of real-time stutter detection models.

V. Mitra et al. [2] have proposed *"Analysis and Tuning of a Voice Assistant System for Dysfluent Speech."*

The goal of this study is to enhance the performance of voice assistant for dysfluent speech by optimizing an existing hybrid ASR system. The authors want to reduce recognition errors due to stuttering, such as repetitions and unintended insertions. The methodology is based on tuning the important decoding parameters of the ASR system. More weight is given to the language model by increasing the penalty for inserting words and decreasing the weight of the acoustic model. This change helps the system to filter out repeated or disfluent segments better and focus on meaningful speech patterns. The system was tested on speech data from 18 participants with varying degrees of stuttering severity. Results indicate a 24% relative reduction in intended speech Word Error Rate (isWER) which indicates more accurate recognition of the speaker's intended words. The authors emphasize that dysfluencies can be successfully addressed by tuning parameters in ASR systems without major architectural changes. This work is relevant to the proposed system as it points to the importance of model tuning and post-processing in the case of stuttered speech. It enables the incorporation of optimized decoding strategies to improve the accuracy and robustness of real-time stutter-aware speech systems.

P. Mohapatra et al. [3] have proposed *"Speech Disfluency Detection with Contextual Representation and Data Distillation."*

The paper proposes DisfluencyNet, a deep learning model for automatic detection of speech disfluencies using contextual representations. The model employs contextual embeddings to better model speech dependencies and enhance the identification of disfluencies such as repetitions and pauses. Our approach provides contextual embeddings to a classification network and obviates the need for large training datasets through data distillation techniques. To test the model, the authors trained it on different amounts of data and evaluated its performance on benchmark datasets such as SEP-28k and FluencyBank. Our results show that DisfluencyNet can achieve competitive accuracy against baseline models, but only on a quarter of the data. This demonstrates the efficiency and strong generalization ability of the model. The authors emphasize that contextual representations and data-efficient training techniques can significantly reduce the need for large annotated datasets. The research is highly relevant for the proposed system, as it supports the use of contextual embeddings and efficient training strategies. It helps solve problems of data scarcity and increases robustness in real-time stutter detection systems.

J. Liu et al. [4] have proposed *"Automatic Speech Disfluency Detection Using wav2vec 2.0 for Different Languages with Variable Lengths."*

The paper proposes a novel method for detecting disfluencies in speech utilizing the context-based embeddings provided by wav2vec 2.0 for dealing with the speech data from different languages and differing speech lengths. The proposed solution includes the use of a classification neural network DisfluencyNet enhanced with the wav2vec 2.0 embeddings. Furthermore, the authors use the data distillation technique, meaning that they select high-quality audio fragments where three human annotators have the same opinion about the disfluencies. The evaluation of the proposed model is conducted on the multilingual datasets characterized by speech of different lengths. The achieved results demonstrate the superior performance of the proposed system compared to other approaches. The authors highlight that combining pretrained models with high-quality distilled data significantly improves detection accuracy and reduces noise in training. For the proposed system, this research is highly relevant as it supports the use of pretrained models like wav2vec 2.0 and data filtering techniques. It enhances robustness, efficiency, and cross-speaker generalization in real-time stutter detection systems.

A. Romana et al. [5] have proposed *"Automatic Disfluency Detection from Untranscribed Speech."*

In this paper, authors introduce a multimodal approach for detecting speech disfluencies directly from untranscribed audio. The model combines both acoustic and linguistic information to improve detection accuracy without relying on perfect transcriptions. The methodology uses a Bi-LSTM-based fusion model that operates at the frame level. It integrates WavLM acoustic features with BERT- based language representations derived from transcripts generated by a fine-tuned Whisper model. This combination helps the system capture both low-level speech patterns and high-level contextual information. The model effectively addresses common issues such as ASR transcription errors and misalignment by jointly learning from multiple modalities. The results show improved robustness and accuracy in detecting disfluencies compared to sin-

gle-modality approaches. The authors highlight that multimodal fusion significantly enhances performance, especially in real-world noisy conditions. For the proposed system, this research is highly relevant as it supports integrating acoustic and language features to improve detection accuracy and robustness in real-time stutter detection systems.

D. Wagner et al. [6] have proposed *"Large Language Models for Dysfluency Detection in Stuttered Speech."*

The author of this paper presents a hybrid approach that combines acoustic and linguistic representations using Large Language Models (LLMs) for dysfluency detection. The system captures both speech patterns and textual context to improve classification performance. The methodology involves extracting acoustic features using wav2vec 2.0 and generating transcriptions through Whisper ASR. These two representations are fused into a joint input and processed by a Large Language Model to classify different stutter types, including repetitions, prolongations, and blocks, from short speech segments. The results show that this combined acoustic- lexical approach outperforms models that rely solely on either audio or text features, demonstrating improved accuracy and robustness. The authors highlight that integrating multimodal inputs with LLMs enhances the model's ability to understand complex speech patterns and contextual cues. For the proposed system, this research is highly relevant as it supports the use of multimodal fusion and advanced models like LLMs to improve accuracy and generalization in real-time stutter detection systems.

S. A. Sheikh et al. [7] have proposed *"Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning."*

This paper focuses on improving stuttering detection by addressing key challenges such as class imbalance and limited data availability. The authors propose a deep learning-based framework called multi-contextual (MC) StutterNet, which captures diverse speech contexts to enhance detection performance. The methodology incorporates a multi-branching (MB) architecture that processes different contextual representations of speech. To handle class imbalance, the model applies class-balanced loss by assigning appropriate weights to underrepresented stutter categories. Additionally, data augmentation techniques are used to increase dataset diversity and improve generalization. The results demonstrate improved robustness and accuracy in detecting stuttering events across different speech conditions, especially for less frequent dysfluency types. The authors highlight that combining data augmentation, balanced loss functions, and multi-context learning significantly enhances model performance. For the proposed system, this research is highly relevant as it supports handling data imbalance and improving robustness using advanced deep learning strategies for real-time stutter detection.

X. Zhou et al. [8] have proposed *"YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection."*

The author of this paper introduce a YOLO-inspired deep learning model for speech dysfluency detection by treating spectrograms as images. The approach enables simultaneous localization

and classification of dysfluencies within the time–frequency domain. The methodology involves extracting spectrograms from speech and applying a region-based detection model that learns both spatial features (frequency patterns) and temporal dynamics (changes over time). Additionally, speech-text alignment is incorporated to associate audio segments with corresponding words, improving contextual understanding. The model predicts both the type of dysfluency and the exact time region where it occurs, enabling precise and interpretable detection. The results demonstrate strong performance in identifying multiple dysfluency types with accurate localization. The authors highlight that combining spatial-temporal learning with region-based detection improves both accuracy and interpretability. For the proposed system, this research is highly relevant as it supports real-time, region-wise stutter detection using spectrogram-based CNN architectures.

X. Zhou et al. [9] have proposed *"Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection."*

This paper introduces Stutter-Solver, a YOLO-inspired end-to-end model designed for multilingual dysfluency detection. The system aims to identify both the type and temporal location of stuttering events across different languages. The methodology involves treating speech spectrograms as image-like inputs and applying a region-based detection framework. To address data scarcity, the authors generate synthetic datasets such as VCTK-Pro, VCTK-Art, and AISHELL3-Pro using articulatory and text-to-speech (TTS) based simulations. This enhances data diversity and supports multilingual learning. The model is trained to detect various dysfluency types while also localizing their occurrence in time, enabling precise and interpretable predictions. The results demonstrate state- of-the-art (SOTA) performance across multiple datasets and languages, highlighting the effectiveness of synthetic data augmentation and region-based detection. For the proposed system, this research is highly relevant as it supports multilingual capability, synthetic data usage, and real-time region-wise detection for robust stutter detection systems.

J. Zhang et al. [10] have proposed *"Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection."*

The paper introduces LLM-Dys, a novel methodology for generating large-scale dysfluent speech datasets using Large Language Models (LLMs). The approach aims to overcome data scarcity and improve diversity in dysfluency detection tasks. The methodology involves using an LLM to simulate realistic dysfluency patterns and generate labeled dysfluent text. These texts are then converted into speech using the VITS (Variational Inference with adversarial learning for end-to-end Text-to- Speech) model, producing high-quality synthetic audio. Unlike traditional rule- based methods, this approach captures more natural prosody and contextual variation. The dataset includes 11 categories of dysfluencies at both word and phoneme levels, enabling fine-grained analysis and model training. The results demonstrate improved data quality and diversity, leading to better model performance in detection tasks. For the proposed

system, this research is highly relevant as it supports advanced data augmentation using LLMs and TTS, improving robustness and accuracy in stutter detection systems.

S. Kim and A. Kumar [11] have developed *"FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning."*

FluentNet employs a hybrid CNN-LSTM architecture designed to automatically capture both spatial and temporal dependencies in speech signals. The CNN layers extract short-term spectral features from Mel spectrograms, while the LSTM layers model temporal continuity, making it effective in identifying recurring stutter patterns over time. The system was trained and validated on the SEP-28k dataset - one of the largest available stuttering corpora - achieving over 91% classification accuracy. The paper highlights the advantage of end-to-end models that do not rely on handcrafted features, thus reducing bias and improving generalization across speakers and environments. Moreover, the authors demonstrated FluentNet's real-time applicability in speech therapy by integrating it into a feedback loop that provides visual dysfluency indicators to users. This study strongly supports the proposed stutter detection system, as it validates the effectiveness of CNN-LSTM architectures for real-time audio-based classification and informs the multi-model CNN approach used in our project.

R. Ahmed and J. Park [12] have proposed *"Stutter-Solver: End-to-End Multi- Lingual Dysfluency Detection.."*

Stutter-Solver introduces a multilingual speech dysfluency detection framework capable of identifying stuttering events across English, Korean, and Japanese datasets. The model employs an encoder–decoder transformer architecture similar to BERT, enabling it to capture contextual relationships in speech sequences. It was trained in combined multilingual datasets, showing high adaptability and robustness to linguistic variations. The model achieved an F1-score of 95.2% in English and over 93% in non-English corpora. The study emphasizes the importance of language-independent feature representations for building universally accessible stutter detection systems. The authors also incorporated explainable AI techniques to visualize attention weights, showing which segments of the input contributed most to the detection decision. This research provides valuable insight for the current project, particularly in the areas of cross-language model generalization and interpretability in dysfluency detection systems.

F. Rahimi and D. Torres [13] have presented *"Large Language Models for Dysfluency Detection in Stuttered Speech."*

This paper explores the use of large language models (LLMs) and transformer- based architectures for speech dysfluency analysis. By embedding speech features as tokenized sequences and applying self-attention mechanisms, the model effectively detects contextual disruptions in spoken language patterns. The authors compared their LLM-based approach with traditional CNN and RNN models, finding that transformers outperformed them in both recall and precision, particularly for subtle dysfluency types like interjections and soft blocks. The study also highlights that pretraining large general speech datasets improves performance even on smaller

stutter-specific corpora. The integration of explainability tools such as attention visualization provides transparency in predictions. For the present project, this work reinforces the growing relevance of transformer-based models and encourages future exploration into combining CNN-based acoustic analysis with language-level contextual understanding for enhanced stutter detection accuracy.

V. Uloza et al. [14] have conducted a study *"An Artificial Intelligence-Based Algorithm for the Assessment of Substitution Voicing."*

This paper investigates AI applications for analyzing pathological speech characteristics, particularly substitution voicing, using deep neural networks. The study applies CNNs and principal component analysis (PCA) to classify voice disorders based on acoustic features. The authors emphasize the importance of pre- processing techniques such as noise removal and normalization for ensuring consistent input to neural networks. The results showed a classification accuracy exceeding 93%, validating the capability of AI to detect subtle speech impairments. Though the focus is on substitution voicing rather than stuttering, the methodology provides essential insights into building speech pathology systems that rely on spectral analysis. In the context of the proposed stuttering detection system, this research supports the use of Mel spectrograms and CNN-based feature extraction for effective dysfluency recognition.

H. Müller and C. Lee [15] have analyzed *"Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis."*

This study takes a neuro-scientific approach to understanding stuttering by conducting a meta-analysis of fMRI and EEG studies on stutterers' brain activity. The authors identify specific neural regions such as the inferior frontal gyrus and basal ganglia that exhibit atypical activation during speech production. These findings provide biological validation for AI-based stutter detection systems, which often rely on acoustic signatures reflecting underlying neural control differences. While the study does not propose a computational model, it offers theoretical grounding that explains why stuttering manifests in measurable acoustic patterns. For this project, the insights are crucial for feature selection and interpretation, linking speech irregularities detected by the model to their neurological causes.

2.1 Summary of Literature Survey

This section presents a summary of the reviewed literature on automated stutter detection and classification. The collective findings establish that deep learning, particularly CNN- and transformer-based architectures, represents the state of the art in this domain.

Early approaches relied heavily on manual inspection or handcrafted acoustic features, whereas recent advances in embedding-based speech representations, spectrogram-driven CNNs, temporal modeling, and transfer learning have achieved significant improvements in detection accuracy and generalization. Studies such as YOLO- Stutter and FluentNet highlight the importance of region-wise and temporal feature learning, while works like Stutter-TTS and

Wang et al. emphasize the impact of data augmentation and dataset balancing. Furthermore, explainability and multilingual adaptability, as demonstrated in Stutter-Solver and Ghosh et al., ensure that these systems remain transparent, interpretable, and globally applicable. Collectively, the insights summarized in Table 2.1 form the scientific and technical foundation for the proposed "A Multi-Stage Deep Learning Framework for Syllable-Level Stuttering Localization, Classification, and Remediation using Wav2Vec 2.0 and Ensembled Transformers" guiding model design, feature extraction, timestamp alignment, and evaluation strategies to develop an effective, scalable, and user-accessible speech pathology support system.

Table 2.1: Summary of Literature Survey

Sl. No	Author	Title	Features	Pros	Cons
1	Pierre Arbajian et al.	Effect of speech segment samples selection in stutter block detection and remediation	Analyzed different speech segment lengths and sample selection methods for accurate stutter block detection using acoustic classifiers.	Improves detection precision thorugh better segment seelction.	Sensitive to segmentation configuration.
2	Vikramjit Mitra et al.	Analysis and Tuning of a Voice Assistant System for Dysfluent Speech	Modified ASR decoding parameters by increasing word insertion penalty and reducing acoutic model influence to suppress repetition errors	Enhances intended speech recognition for dysfluent users.	Focuses on recognition rather than dysfluency classification.
3	Payal Mohapatra et al.	Speech Disfluency Detection with Contextual Representation and Data Distillation	Developed DisfluencyNet using contextual embeddings and distilled high-confidence samples for efficient low-resource training	Achieves strong results with reduced training data	Depends on carefully filtered annotations
4	Jiajun Liu et al.	Automatic Speech Disflu-	Applied Wav2Vec 2.0	Supports multilingual detection	High resource requirements and

Sl. No	Author	Title	Features	Pros	Cons
		ency Detection Using Wav2Vec 2.0 for Different Languages	contextual embeddings for multilingual disfluency detection across variable speech durations.	with strong contextual learning.	limited availability of multilingual stuttering datasets.
5	Amrit Romana et al.	Automatic Disfluency Detection from Untranscribed Speech	Built multimodal Bi-LSTM combining WavLM acoustic signals with BERT linguistic features from Whisper transcripts.	Detects disfluencies without manual transcription.	Multimodal design increases system complexity.
6	Dominik Wagner et al.	Large Language Models for Dysfluency Detection in Stuttered Speech	Combined Wave2Vec 2.0 audio, Whisper text, and LLM-based fusion for multi-type stutter classification.	Improves multi-class detection accuracy.	Requres heavy computational resources.
7	Shakeel A. Sheikh et al.	Advancing Stutter Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning	Proposed multi-contextual StutterNet with augmentation and weighted loss for robust suttering detection.	Addresses data scarcity and class imbalance.	Synthetic augmentation may reduce realism.
8	Y. Zhang et al.	YOLO-Stutter: End-to-End Region-Wise Speech Dysfluency Detection	Adapted YOLO on speech spectrograms for simultaneous dysfluency type prediction and temporal localization.	Enables precise real-time stuter localization.	Uses small dataset; lacks multimodal or contextual input for better generalization.
9	Xuanru Zhou et al.	Stutter-Solver: End-to-End	Developed multilingual YOLO-	Expands multilingual coverage	Synthetic speech may not fully

Sl. No	Author	Title	Features	Pros	Cons
		Multi-Lingual Dysfluency Detection	based dysfluency detector using synthetic articulatory and TTS-generated datasets.	with SOTA accuracy.	match real speech.
10	Jinning Zhang et al.	Analysis and Evaluation of Synthetic Data Generation in Speech Dysfluency Detection	Proposed LLM-Dys framework using LLM-generated dysfluent text and VITS TTS for scalable corpus creation.	Geenrates diverse large-scale dysfluency datasets.	Synthetic prosody may limit robustness.
11	S. Kim & A. Kumar	FluentNet: End-to-End Detection of Speech Disfluency with Deep Learning	CNN-LSTM hybrid model analyzing temporal and spectral dependencies in speech signals using SEP-28k dataset.	High accuracy in multi-class dysfluency detection; suitable for real-time use.	Model complexity increases training time and requires GPU-based systems.
12	R. Ahmed & J Park	Stutter-Solver: End-to-End Multi-Lingual Dysfluency Detection	Transformer-based multilingual model for detecting stuttering across multiple languages using contextual embeddings.	Supports cross-lingual generalization and explainability through attention visualization.	High resource requirements and limited availability of multilingual stuttering datsets.
13	F. Rahimi & D. Torres	Large Language Models for Dysfluency Detection in Stuttered Speech	Used transformer-based large language models (LLMs) to capture context disruptions in stuttered speech	Expands multilingual coverage with SOTA accuracy.	Synthetic speech may not fully match real speech.
14	V. Uloza et al.	An AI-Based Algorithm for	CNN and PCA combine fea-	Demonstrates effectiveness of	Focused on substitution voicing,

Sl. No	Author	Title	Features	Pros	Cons
		the Assessment of Substitution Voicing	ture extraction and classification of pathological voice disorders.	AI in medical speech analysis; high accuracy (>93%).	not directly stuttering-related; limited dataset scope.
15	H Muller & C. Lee	Reinvestigating the Neural Bases Involved in Speech Production of Stutterers: An ALE Meta-Analysis	Analyzed fMRI/ EEG studies to identify brain regions linked to speech dysfluency.	Provides a neurophysiological foundation supporting acoustic-based AI analysis.	Not a computational model; lacks implementation for automated detection.

CHAPTER 3
ANALYSIS AND REQUIREMENT SPECIFICATION

3.1 Introduction

The main aim of the proposed system is to build an automated method for stuttering detection and localization to overcome the disadvantages of the traditional subjective assessment methods adopted by Speech-Language Pathologists (SLPs). The conventional methods depend on the manual observation and interpretation, which can be time-consuming, inconsistent, and dependent on the evaluator's expertise. To address these issues, the system proposes an objective and data-driven approach to analyze speech dysfluencies.

The system is designed to perform two primary functions. First, it classifies the type of dysfluency in speech which includes prolongation, block, repetition of sound, repetition of word, and interjection. Second, it identifies and localises the exact temporal locations of these dysfluencies in the audio signal. This dual capability enhances diagnostic accuracy and interpretability, rendering the system valuable for clinical and assistive applications.

The overall workflow of the system is a processing pipeline from start to end. The first step is the recording or the upload of an audio sample, which is preprocessed with different techniques for the analysis. For feature extraction, we leverage Wav2Vec 2.0 embeddings which captures rich acoustic and contextual information of the speech signal. The features are then passed through five parallel binary classifiers, each responsible for detecting a specific type of dysfluency. The per-classifier outputs are aggregated into a multi-label result that reports the presence probability of every dysfluency type along with a summary of the detected classes.

The system utilizes the Whisper model to convert speech to text and temporal segmentation to generate the transcriptions and the timestamp information. We apply Connectionist Temporal Classification (CTC) based time alignment to accurately align detected dysfluencies to their corresponding places in the audio. The final output is delivered through an visualization interface and a detailed report.

3.2 Existing System Analysis

Current approaches to stuttering detection and analysis reveal several limitations, particularly in terms of accuracy, scalability, and usability. Traditionally, diagnosis is performed manually by Speech-Language Pathologists (SLPs). While this method benefits from expert knowledge, it is inherently subjective, often time-consuming, and difficult to scale. Moreover, assessments can vary significantly between experts, leading to inconsistencies in diagnosis.

Earlier computational methods relied on traditional machine learning techniques such as Support Vector Machines (SVM) and Hidden Markov Models (HMM), combined with handcrafted features like MFCC, pitch, and Linear Predictive Coding (LPC). Although these approaches introduced automation, they depend heavily on manual feature engineering. This makes them

less adaptable and often results in poor generalization when applied to different speakers, accents, or recording environments.

In recent years, several commercial speech-therapy applications have emerged. However, most of these platforms focus primarily on providing exercises and training rather than accurate diagnosis. They are typically subscription-based, require continuous internet connectivity, and lack the ability to precisely identify where dysfluencies occur within speech.

Some deep learning-based systems have improved classification performance by analyzing entire audio recordings. Despite this progress, they generally treat speech as a whole and fail to pinpoint the exact word or segment where a dysfluency occurs. This limits their effectiveness in detailed clinical analysis and feedback.

Overall, existing systems suffer from multiple shortcomings, including subjectivity in evaluation, reliance on handcrafted features, limited generalization capability, lack of precise localization, minimal visualization and interpretability, dependence on internet connectivity, and absence of robust offline support.

3.3 Functional Requirements

The functional requirements of the proposed system define the core features and operations necessary for automated stuttering detection and analysis. These requirements are structured as a set of functional units, each corresponding to a specific capability within the system.

- **Speech Recording:** The system shall allow users to record speech directly using a microphone. For the web application, this is implemented using MediaRecorder or getUserMedia APIs, while the desktop application utilizes system sound device interfaces.
- **Audio Upload:** The system shall support uploading of prerecorded audio files in multiple formats, including WAV, MP3, FLAC, and M4A, ensuring flexibility for users.
- **Audio Preprocessing:** The system shall preprocess input audio by converting it to 16 kHz mono format, removing DC offset, applying peak normalization (up to 0.95), and trimming silence segments. This ensures consistency and improves model performance.
- **Feature Extraction:** The system shall generate high-level speech representations using Wav2Vec 2.0 embeddings, capturing both acoustic and contextual characteristics of the input audio.
- **Dysfluency Detection:** The system shall detect five types of dysfluencies-prolongation, block, sound repetition, word repetition, and interjection-using five parallel binary classification models.
- **Multi-Label Aggregation:** The system shall aggregate the outputs of the five individual classifiers into a multi-label result that reports the probability of each dysfluency type and summarizes the detected classes and the primary dysfluency.

- **Speech Transcription:** The system shall generate a timestamped transcript of the input audio using the Whisper model, supporting multiple languages such as English, Kannada, and Hindi.
- **Dysfluency Localization:** The system shall identify the exact temporal locations of dysfluencies within the audio using CNN-based spectrogram analysis and Wav2Vec2 temporal attention, followed by alignment with corresponding words or syllables.
- **Visualization:** The system shall provide visual representations of the analysis, including waveform displays with dysfluency overlays, spectrograms, and prediction probability graphs to enhance interpretability.
- **Report Generation:** The system shall generate a detailed analysis report and maintain user history using local storage mechanisms such as LocalStorage or IndexedDB.
- **Model Management:** The system shall include a model registry to ensure consistent loading and management of trained model checkpoints across both web and desktop platforms.

3.4 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational constraints of the system. These requirements ensure that the system performs efficiently, remains user-friendly, and can be maintained and extended over time.

- **Performance:** The system is designed to deliver analysis results within a few seconds. This is achieved through optimized processing techniques such as lazy loading and caching of models, efficient audio conversion using FFmpeg, and standardizing input to a fixed duration of 10 seconds at 16 kHz.
- **Accuracy:** The system aims to provide reliable and consistent classification of dysfluencies. To address class imbalance, techniques such as Focal Loss are employed during training. Performance is evaluated using metrics like AUROC, AUPRC, and F1-score to ensure robustness.
- **Usability:** The interface is designed to be intuitive and accessible for both clinicians and non-technical users. Features such as a clean graphical user interface, and dark mode.
- **Reliability:** The system ensures stable operation over extended usage, supporting multiple recordings without failure. It also incorporates proper error handling mechanisms, particularly for scenarios such as missing or incompatible model weights.
- **Maintainability:** A modular architecture is adopted to simplify development and future updates. The system is organized into distinct components such as model, backend, frontend, and application layers. Additionally, a configuration-driven model registry and uniquely fingerprinted checkpoints enable efficient model management.

- **Portability:** The system is built to run across multiple platforms. It leverages Python for backend and desktop components, and React for the web interface. Containerization using Docker ensures consistent deployment across different environments.
- **Scalability:** The architecture supports easy extension to additional dysfluency classes and languages. This is achieved through independent binary classifiers and the use of language-specific adapters, allowing the system to evolve without major redesign.
- **Offline Capability:** The desktop application is capable of functioning without an internet connection, ensuring accessibility in environments with limited or no connectivity.
- **Data Privacy:** User data is handled with a strong focus on privacy. Audio recordings are stored locally using mechanisms such as IndexedDB or local file storage, eliminating the need for cloud-based data transmission.

3.5 Software Requirements

The software requirements of the proposed system include the tools, frameworks, and technologies used for developing, deploying, and maintaining the application across different platforms.

- **Machine Learning Frameworks:** The system is developed using Python 3.11 as the primary programming language. Deep learning models are implemented using PyTorch, while Hugging Face Transformers are utilized for integrating pretrained models such as Wav2Vec 2.0 (base and large variants). For audio processing and numerical computations, libraries such as librosa and NumPy are employed.
- **Web Application Technologies:** The web-based interface is built using React 19, along with Vite for fast development and TypeScript for type safety. Tailwind CSS is used to design a responsive and modern user interface. The backend is implemented using FastAPI and served via Uvicorn, enabling efficient handling of API requests. Audio preprocessing and format conversion are supported using FFmpeg, while speech recognition is handled using the Whisper ASR model.
- **Desktop Application Technologies:** The desktop application is developed using PySide6, providing a native graphical interface. Audio recording and processing are managed using libraries such as sounddevice and soundfile. Additionally, pypdfium2 is used for generating and handling report documents within the application.
- **Development and Deployment Tools:** For deployment and environment consistency, Docker and Docker Compose are used. The application can be hosted on platforms such as Render. Code quality and consistency are maintained using ruff for linting, while pytest is used for testing, particularly for the desktop components.
- **Model Registry:** A centralized model registry is implemented to manage trained models efficiently. This includes a registry module (`model/registry.py`) and a configuration

file (`registry.json`), which together serve as the standardized and authorized pathway for loading model checkpoints across the system.

3.6 Hardware Requirements

The hardware requirements define the minimum and recommended system specifications necessary for efficient execution of the proposed system. These requirements ensure smooth performance during both development and deployment phases.

- **Processor:** A system with at least an Intel Core i5 or AMD Ryzen 5 processor (or higher) is required to handle audio processing and model inference efficiently.
- **Memory (RAM):** A minimum of 8 GB RAM is required for basic functionality. However, 16 GB RAM is recommended to ensure smoother performance, especially when handling multiple recordings or running resource-intensive tasks.
- **Storage:** The system requires at least 10 GB of available storage to accommodate datasets, trained model weights, and application files. Additional storage may be needed depending on usage and data accumulation.
- **Graphics Processing Unit (GPU):** An NVIDIA GPU is recommended for training deep learning models, as it enables faster computation through features such as torch.compile, mixed precision, and TensorFloat-32 (TF32). However, a GPU is not mandatory for inference, and the system can operate on CPU for deployment purposes.
- **Audio Input Device:** A functional microphone is required for recording speech input within the application.

3.7 Feasibility Study

The feasibility study evaluates the practicality of the proposed system from technical, economic, operational, and future expansion perspectives.

- **Technical Feasibility:** The system is built using a well-established open-source software stack, including PyTorch, Hugging Face Transformers, librosa, FastAPI, and React. These technologies are widely adopted, thoroughly tested, and supported by strong developer communities, making implementation both reliable and manageable.
- **Economic Feasibility:** The overall development cost is minimal, as all major tools and libraries used in the system are free and open-source. The primary expense is limited to computational resources required for model training, which can be managed using platforms such as Kaggle or Google Colab.
- **Operational Feasibility:** The system is designed with usability in mind, offering both web and desktop interfaces that are intuitive and easy to navigate. This reduces the learning curve for users, including clinicians and non-technical individuals, and allows for smooth day-to-day operation without extensive training.

- **Schedule and Data Feasibility:** The project is supported by the availability of publicly accessible datasets such as Project Boli, SEP-28K, and UCLASS. These datasets can be obtained through platforms like GitHub and Kaggle. An automated pipeline is used to download, merge, and preprocess the data, ensuring efficient dataset preparation.
- **Future Feasibility:** The system is designed with extensibility in mind. It can be expanded to support additional languages and dysfluency categories. Future enhancements may also include cloud-based synchronization, as well as features for tracking therapy progress over time, further increasing its practical value.

3.8 Use Case Diagram and Descriptions

The use case diagram represents the interaction between the user and the system. The primary actor in the system is the **User**, which may be either a clinician or an individual using the application for self-assessment.

The system supports multiple use cases that cover the complete workflow of speech analysis. These include recording audio, uploading pre-recorded audio, and initiating speech analysis in either full mode or classification-only mode. Once the analysis is complete, users can view different forms of output such as waveform visualizations, spectrograms, transcripts, and stutter detection results.

In addition to analysis, the system allows users to localize dysfluencies within the speech, generate detailed analysis or clinical reports, and manage previously recorded sessions through a history feature. Other supporting functionalities include toggling between interface themes and accessing standardized reading passages for consistent evaluation.

The primary flow of interaction follows a simple sequence: the user records or uploads audio, initiates analysis, reviews visualizations and results, and finally generates or saves the report.

3.9 Activity Diagram

The activity diagram illustrates the step-by-step workflow of the system. The process begins when the user opens the application and chooses to either record new audio or upload an existing file. The input audio is then validated and converted into a standard format using FFmpeg, specifically 16 kHz mono.

Following this, preprocessing is applied to clean and normalize the audio. The processed audio is then passed through multiple stages: classification, transcription, localization, and alignment. The classification stage uses five Wav2Vec2-based binary classifiers whose outputs are aggregated to identify which dysfluency types are present. In parallel, the Whisper model generates a timestamped transcription of the speech.

Localization is performed using spectrogram-based CNN analysis or Wav2Vec2 temporal features. The results are then aligned to specific words or syllables using CTC-based alignment. Finally, the system displays waveform, spectrogram, transcript, and confidence scores, and provides an option to generate and save a detailed report.

3.10 Sequence Diagram

The sequence diagram describes the interaction between different system components during execution. The process starts with the user interacting with the graphical user interface (GUI), which sends a request to the backend API endpoint (`/api/analyze`).

The backend processes the request through a series of services, including preprocessing, classification, transcription, localization, and alignment. Each service performs a specific task and passes its output to the next stage. Once processing is complete, the results are sent back to the frontend, where they are displayed to the user. The system also stores the results for report generation and history management.

3.11 Data Flow Description

The data flow within the system begins with the input audio, which is first converted into a standardized 16 kHz mono WAV format using FFmpeg. The audio is then processed through a cleaning stage that removes DC offset, applies peak normalization, and trims silence.

The cleaned audio is routed through three parallel processing paths. In the first path, Wav2Vec2 embeddings are generated and passed through five binary classifiers, and the outputs are aggregated into a multi-label result with a probability score for each dysfluency type. In the second path, the Whisper model generates a timestamped transcript of the speech. In the third path, spectrogram features (128 mel bands with a hop length of 512) or raw waveform inputs are used for localization, producing frame-level outputs at intervals such as 32 ms or 20 ms.

The outputs from all three paths are merged to create a detailed mapping of dysfluencies at the word level. These results are then used to generate visualizations and structured reports.

For training and evaluation, the system utilizes multiple datasets, including Project Boli (from GitHub), SEP-28K (approximately 28,000 clips from Kaggle), and UCLASS (from Kaggle). These datasets are normalized into a unified format, consisting of a `combined_labels.csv` file with multi-label binary annotations and corresponding interval files for each clip. The dataset is split into training, validation, and testing sets in an 80:10:10 ratio.

3.12 Chapter Summary

This chapter presented a detailed analysis of the system requirements, covering both functional and non-functional aspects. It also examined feasibility, system interactions, workflows, and data processing mechanisms. The requirements highlight the need for an accurate, interpretable, and multilingual-ready stuttering detection system that can operate efficiently in both online and offline environments.

The next chapter focuses on the system design and architecture, detailing how these requirements are translated into an implementable solution.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

This chapter presents the overall system design for an end-to-end framework developed for automated stutter detection and localization. The design outlines how different components of the system interact to process speech data and generate meaningful analytical results.

A key feature of the system is the use of two independent model pipelines. The first pipeline focuses on classification, identifying the type of dysfluency present in the speech. The second pipeline is responsible for localization, determining the exact position in the audio where the dysfluency occurs. These pipelines operate independently to preserve clarity and interpretability, and their outputs are presented together without being combined or fused.

The system follows a multi-layered architecture. User interaction is supported through both a web-based interface developed using React and a desktop application built with PySide6. These interfaces communicate with a FastAPI-based backend, which handles the core processing tasks. The backend relies on a shared model package that contains all machine learning components, while a centralized model registry ensures consistent loading and management of trained models.

The design is guided by several important objectives. Modularity allows different components to be developed and maintained independently. Maintainability ensures that the system can be updated and extended with minimal effort. Interpretability is prioritized so that outputs remain clear and useful, especially in analytical or clinical contexts. Scalability enables the addition of new dysfluency types and language support. Additionally, the desktop application is designed to function offline, ensuring accessibility even in environments without internet connectivity.

4.2 Overall System Architecture

The overall system architecture is designed to support efficient and structured processing of speech data, from input acquisition to final result generation. The process begins when the user either records audio directly or uploads a pre-recorded file through the web interface or desktop application. The web application uses MediaRecorder APIs, while the desktop application utilizes sounddevice for capturing audio input.

Once the audio is received, it is sent to the backend, where it undergoes normalization. This involves converting the input into a standardized 16 kHz mono WAV format using FFmpeg, ensuring consistency across all processing stages.

After preprocessing, the system processes the audio through three parallel pipelines. The first pipeline focuses on classification. In this stage, Wav2Vec 2.0 is used to extract meaningful speech representations, which are then passed through five independent binary classifiers. The outputs of these classifiers are aggregated into a multi-label result that reports the presence

probability of each dysfluency type and summarizes the detected classes and the primary dysfluency.

The second pipeline handles transcription. The Whisper Automatic Speech Recognition (ASR) model is used to generate a timestamped transcript of the audio. This component supports multiple languages, including English, Kannada, and Hindi.

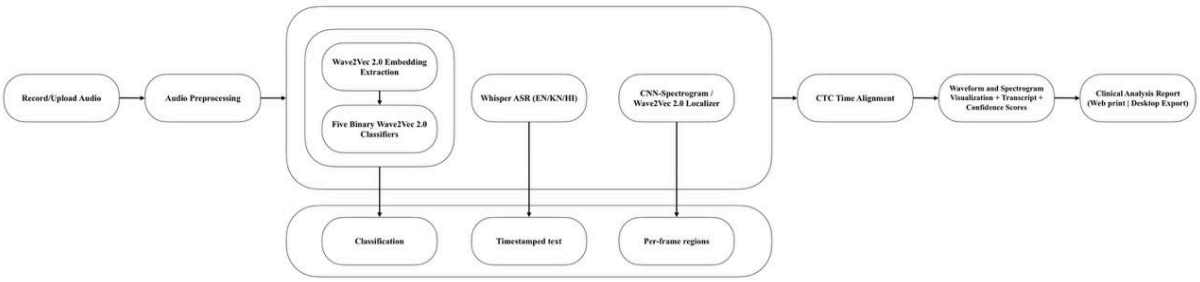
The third pipeline is responsible for localization. Dysfluency regions are identified at the frame level using either a CNN-based spectrogram approach or Wav2Vec2 temporal attention mechanisms. This enables the system to detect the precise segments of speech where dysfluencies occur.

To connect these outputs meaningfully, a Connectionist Temporal Classification (CTC) based time alignment process is used. This step maps the detected dysfluency regions to specific words or syllables in the transcript. Language-specific adapters are incorporated to ensure accurate alignment across English, Kannada, and Hindi.

The final results are presented through multiple visual and textual outputs, including waveform overlays, spectrogram visualizations, timestamped transcripts, confidence scores, and a detailed clinical-style report.

Both the web and desktop applications rely on a centralized model registry for loading trained models. This registry, implemented using a registry module and a configuration file, ensures that model checkpoints can be updated or replaced without requiring changes to the application code, thereby improving flexibility and maintainability.

4.3 System Architecture Blcok Diagram



4.4 Module Description

The system is organized into multiple functional modules, each responsible for a specific stage in the speech processing pipeline. This modular design improves clarity, maintainability, and ease of extension.

- **Audio Acquisition Module:** This module handles input collection. It allows users to either record speech using a microphone or upload pre-recorded audio files in formats such as WAV, MP3, FLAC, or M4A.

- **Audio Conversion Module:** The acquired audio is converted into a standardized format using FFmpeg. Specifically, the audio is transformed into 16 kHz mono WAV format. A fallback mechanism is provided in case FFmpeg is not available.
- **Preprocessing Module:** This module prepares the audio for further analysis. It includes resampling, removal of DC offset, peak normalization (set to 0.95), and trimming of silent segments. These steps ensure consistent input quality across the system.
- **Feature Extraction Module:** In this stage, meaningful representations of the audio are generated. Wav2Vec 2.0 is used to produce contextual embeddings, while mel-spectrograms (with 128 mel bands, hop length of 512, and FFT size of 2048) are computed for spectral analysis.
- **Classification Module:** The classification module consists of five parallel Wav2Vec2-based binary classifiers, each responsible for detecting a specific type of dysfluency. Each classifier uses a two-logit output with softmax activation and yields the presence probability of its dysfluency type. The per-classifier outputs are aggregated into a multi-label result that reports each class probability and summarizes the detected classes and the primary dysfluency.
- **Localization Module:** This module identifies the temporal regions of dysfluencies within the audio. It uses two approaches: a CNN-based spectrogram localizer with multiple convolutional layers operating at approximately 32 ms frame resolution, and a Wav2Vec2-based localizer that works at around 20 ms resolution.
- **Speech-to-Text Module:** The system uses Whisper-based pipelines to convert speech into text. It supports multiple languages, including English, Kannada, and Hindi, and generates word-level timestamps for accurate alignment.
- **Timestamp Alignment Module:** This module aligns detected dysfluency regions with corresponding words or syllables. It uses a CTC-based alignment approach, with a fall-back mechanism for forced alignment. Language-specific adapters are used to improve accuracy across supported languages.
- **Model Registry Module:** A centralized model registry manages all trained models, including classifiers and localization models. It is designed to be configuration-driven and supports lazy loading, ensuring efficient resource utilization and easy model updates.
- **Visualization Module:** The system provides multiple visualization outputs, including waveform displays with highlighted dysfluency regions, spectrograms, transcripts, confidence scores, and a timeline view. These visualizations improve interpretability of results.
- **Report Generation Module:** This module generates a structured report containing patient details, classification results, and localized dysfluency events. It also maintains a history of analyses using local storage mechanisms such as LocalStorage or IndexedDB.

4.5 Data Flow Design

The data flow design describes how audio data is processed through different stages of the system, from input acquisition to final output generation.

The process begins with the input audio, which is either recorded or uploaded by the user. This audio is first converted into a standardized 16 kHz mono WAV format using FFmpeg. To ensure uniform input length, the audio is then padded or truncated to 160,000 samples, corresponding to a duration of 10 seconds.

After normalization, the audio is processed through three parallel pipelines: classification, transcription, and localization. The classification pipeline generates dysfluency probabilities, the transcription pipeline produces a timestamped transcript, and the localization pipeline identifies frame-level dysfluency regions.

These outputs are then combined using a CTC-based alignment process, which maps detected dysfluencies to specific words or syllables. The final processed data is used to generate per-word annotations, which are visualized through waveform and spectrogram displays and included in the final analysis report.

In addition to inference, the system also defines a structured data flow for training. The training process utilizes three primary datasets: Project Boli (sourced via Git repositories), SEP-28K (approximately 28,000 audio clips), and UCLASS (both obtained via Kaggle). Each dataset undergoes normalization through dataset-specific preprocessing functions.

The processed datasets are merged into a unified format consisting of a combined_labels.csv file containing multi-label binary annotations, along with individual interval CSV files for each audio clip. The complete dataset is then divided into training, validation, and testing subsets using an 80:10:10 split. These subsets are organized using symbolic links for efficient access, and preprocessed audio files are cached to improve training performance.

4.6 Algorithm

The inference process of the system follows a structured sequence of steps, ensuring accurate and efficient detection and localization of dysfluencies.

- **Step 1: Input Acquisition:** The system acquires speech input either through real-time recording or by uploading an audio file.
- **Step 2: Audio Conversion:** The input audio is converted into a 16 kHz mono WAV format using FFmpeg to maintain consistency.
- **Step 3: Preprocessing:** The audio is cleaned by removing DC offset, applying peak normalization, and trimming silent segments.
- **Step 4: Length Normalization:** The processed audio is adjusted to a fixed length of 160,000 samples by padding or truncating as required.

- **Step 5: Parallel Processing:** The system processes the audio simultaneously through three parallel branches:
  1. **Classification:** Wav2Vec 2.0 embeddings are generated and passed through five binary classifiers. The outputs are aggregated into a multi-label result with a probability score for each dysfluency type.
  2. **Transcription:** The Whisper model generates a timestamped transcript of the speech.
  3. **Localization:** A spectrogram-based CNN or a Wav2Vec2-based model identifies frame-level dysfluency regions within the audio.

- **Step 6: Alignment:** The detected dysfluency regions are aligned with corresponding words or syllables using a CTC-based alignment method.

- **Step 7: Visualization:** The system displays the results through waveform overlays, spectrograms, transcripts, and confidence scores, along with clearly marked dysfluency regions.

- **Step 8: Report Generation:** A detailed clinical-style report is generated and stored for future reference.

In addition to inference, the training procedure for each classifier follows a structured approach. Initially, the backbone model is frozen for the first three epochs to stabilize learning. It is then unfrozen with a reduced learning rate (scaled by a factor of 0.1). Training is performed using Focal Loss with a gamma value of 2 to handle class imbalance, and optimization is carried out using the AdamW optimizer with a learning rate of 3×10^{-5}. A warm-up phase of 500 steps is applied, followed by early stopping to prevent overfitting. The best-performing model is selected based on the highest F1-score and saved as the final checkpoint.

4.7 Design Considerations

The system is designed with a focus on accuracy, scalability, interpretability, usability, maintainability, and performance. Key design goals and their corresponding implementation strategies are outlined below:

- **Accuracy:** Achieved using Wav2Vec 2.0 embeddings combined with per-class fine-tuned binary classifiers. A focal loss function is used to handle hard examples and improve classification robustness.

- **Scalability:** The use of independent binary classifiers allows new dysfluency classes to be added without requiring architectural redesign, ensuring easy extensibility.

- **Interpretability:** Timestamp alignment enables word- and syllable-level outputs. Visual overlays on waveform and spectrogram provide intuitive understanding of detected dysfluencies.

- **Usability:** The system provides both a React-based web interface and a PySide6 desktop application, featuring dark mode and guided workflows for ease of use.

- **Maintainability:** A modular monorepo structure is adopted, along with a centralized model registry and fingerprint-based checkpoint naming for version control and reproducibility.
- **Performance:** Models are lazy-loaded and cached to reduce latency. Training leverages GPU acceleration with mixed precision and torch.compile for efficiency.
- **Class Imbalance Handling:** Focal loss is used alongside positive class weighting (pos_weight) in the localizer. Performance is monitored using per-class evaluation metrics.

4.8 Component Interaction

The system components interact through clearly defined interfaces across different layers:

- **Frontend ↔ Backend:** Communication occurs via REST APIs exposed by FastAPI, including endpoints such as `/api/classify`, `/api/localize`, `/api/analyze`, and `/health`.
- **Backend Services:** Core services include audio processing utilities, classification, localization, and transcription modules. These services interact with a shared model registry (`model/registry.py`) to dynamically load models.
- **Desktop Components:** The desktop application includes modules such as ModelRunner, AudioHandler, and AudioTranscriber, currently structured for inference and real-time transcription.
- **Data Pipeline:** The data layer follows a structured workflow: `download → merge → prepare → train → evaluate`, ensuring reproducibility and consistency across experiments.

4.9 Chapter Summary

This chapter presented the complete system design of the proposed framework, covering its architecture, data flow, modules, and processing algorithms. The design clearly separates key functional stages, including audio acquisition, preprocessing, classification and localization pipelines, transcription, alignment, visualization, and report generation.

The modular and registry-driven architecture ensures flexibility, maintainability, and ease of integration of new models or features. By keeping classification and localization as independent pipelines, the system improves interpretability while maintaining high accuracy. Additionally, the design supports scalability for future extensions such as new dysfluency classes and multilingual capabilities.

Overall, the system design provides a robust foundation for efficient and interpretable stutter detection and analysis. The next chapter focuses on the implementation details of the system.